

CHAPTER 3

Python Fundamentals

Data Types · Variables · Control Flow · Data Structures · OOP · Exception Handling

Learning Objectives

By the end of this chapter, you will be able to:

- Identify and work with Python's core built-in data types.
- Declare, assign, and manipulate variables effectively.
- Write interactive programs using `input()` and `print()`.
- Convert between data types safely using type conversion functions.
- Implement decision logic using `if`, `elif`, and `else` statements.
- Apply comparison and logical operators to form conditions.
- Store and manipulate data using lists, dictionaries, and sets.
- Iterate over data using `for` loops, `while` loops, and the `range()` function.
- Design classes and objects using Object-Oriented Programming principles.
- Handle runtime errors gracefully using `try-except-finally` blocks.

SECTION 1 — THEORETICAL FOUNDATIONS

1.1 Data Types

In Python, a **data type** categorizes the kind of value a variable can hold and determines what operations can be performed on it. Python is a **dynamically typed** language, meaning you do not need to explicitly declare a variable's type — the interpreter infers it automatically from the value assigned. This is in contrast to statically typed languages such as Java or C++, where the type must be declared before use.

The four fundamental data types in Python that every programmer must master are:

Data Type	Description, Characteristics & Examples
<code>int</code>	— Integer. Represents whole numbers with no decimal component. Integers in Python have arbitrary precision — they can be as large as memory allows. Examples: 0, 10, -25, 1_000_000 (underscores improve readability).
<code>float</code>	— Floating-Point Number. Represents real numbers with a fractional part, stored using IEEE 754 double-precision format. Examples: 3.14, -0.001, 2.0, 1.5e10 (scientific notation). Note: floats may have small precision errors due to binary representation.
<code>str</code>	— String. An immutable sequence of Unicode characters. Strings can be enclosed in single quotes, double quotes, or triple quotes (for multi-line strings). Examples: 'hello', "Python", "multi\nline".
<code>bool</code>	— Boolean. Represents one of two logical values: True or False. Booleans are a subclass of int in Python (True == 1, False == 0). They are the result of comparison and logical operations.

Key Concept: Dynamic Typing

Python determines a variable's type at runtime based on the value assigned to it. The built-in function `type()` can be used to inspect the data type of any variable or expression. For example, `type(42)` returns `<class 'int'>`, and `type(3.14)` returns `<class 'float'>`.

1.2 Variables

A **variable** is a named reference to a location in memory where a value is stored. Variables act as containers — they allow programmers to assign meaningful names to data so that it can be referenced, manipulated, and reused throughout a program. In Python, variable assignment is performed using the `=` operator.

Python's rules and conventions for variable naming are:

- Variable names may contain letters (a–z, A–Z), digits (0–9), and underscores (`_`).
- Variable names must **begin with a letter or underscore** — not a digit.
- Python is **case-sensitive**: `age`, `Age`, and `AGE` are three distinct variables.
- Reserved keywords (e.g., `if`, `for`, `class`) cannot be used as variable names.
- By convention, variable names use **snake_case** (e.g., `total_price`, `user_name`). This is specified in PEP 8, Python's official style guide.

Best Practice: Meaningful Names

Variable names should clearly describe the data they hold. `x = 85` is legal but uninformative. `student_score = 85` is self-documenting and dramatically improves code readability, especially in large Data Science projects where notebooks may have hundreds of variables. Always write code as if another developer — or your future self — will read it.

1.3 Input and Output

Interaction between a program and its user is achieved through two fundamental operations: **input** (reading data from the user) and **output** (displaying data to the user). Python provides built-in functions for both.

1.3.1 The print() Function

The `print()` function writes the specified values to the standard output (typically the terminal or console). It accepts multiple arguments separated by commas, an optional `sep` parameter (specifying the separator between values, default is a space), and an optional `end` parameter (specifying what to print at the end, default is a newline `\n`).

1.3.2 The input() Function

The `input()` function pauses program execution and waits for the user to type a value and press Enter. The function accepts an optional **prompt string** argument that is displayed to the user before input is collected. Critically, `input()` **always returns a string** — regardless of what the user types. If you enter `42`, the function returns the string `"42"`, not the integer `42`. This makes type conversion an essential companion skill.

1.4 Type Conversion

Type conversion, also known as **type casting**, is the process of transforming a value from one data type into another. Because `input()` always returns a string, any numeric computation on user-supplied data requires explicit conversion. Python provides a family of built-in conversion functions for this purpose.

Function	Converts To & Behaviour
<code>int(x)</code>	Converts x to an integer. Works on numeric strings ("42" → 42) and floats (3.9 → 3, truncates). Raises ValueError on non-numeric strings.

<code>float(x)</code>	Converts <code>x</code> to a float. Works on integer strings ("3" → 3.0) and integers (5 → 5.0). Raises <code>ValueError</code> on non-numeric strings.
<code>str(x)</code>	Converts <code>x</code> to a string representation. Works on any data type. Always succeeds. Useful for concatenating non-string values.
<code>bool(x)</code>	Converts <code>x</code> to a Boolean. Falsy values (0, 0.0, "", None, [], {}, set()) → False. All other values → True.

Important: Implicit vs. Explicit Conversion

Python performs some conversions automatically (**implicit conversion**), such as promoting an `int` to `float` in a mixed arithmetic expression (`3 + 1.5 = 4.5`). However, most conversions require an explicit function call. Always validate user input before conversion to avoid `ValueError` exceptions — for example, check that a string is numeric before calling `int()` on it.

1.5 Conditional Statements

Conditional statements give a program the ability to **make decisions** — to execute different blocks of code depending on whether certain conditions are true or false. This transforms a program from a simple sequence of instructions into a dynamic decision-making system. Python provides three keywords for conditional logic: `if` , `elif` (else-if), and `else` .

Python uses **indentation** (typically 4 spaces) — not curly braces — to define code blocks. Any code that belongs to a conditional branch must be consistently indented beneath it. Incorrect indentation is a syntax error in Python.

1.5.1 The if Statement

The `if` statement is the most basic form of conditional logic. It evaluates a Boolean expression — the **condition** — and executes the indented code block only if the condition evaluates to `True` . If the condition is `False` , the block is skipped entirely and program execution continues at the next unindented statement.

1.5.2 The if-elif-else Statement

When more than two mutually exclusive conditions exist, the `elif` (else-if) clause allows additional conditions to be checked in sequence. The `else` clause, if present, executes its block only when all preceding `if` and `elif` conditions are `False` . Python evaluates conditions **top to bottom** and executes only the **first matching block** — subsequent blocks are skipped.

1.6 Comparison and Logical Operators

Comparison operators evaluate the relationship between two values and return a Boolean result. They are the primary building blocks of conditional expressions in Python.

Operator	Name & Behaviour
<code>==</code>	Equality. Returns True if both operands are equal in value. Note: <code>==</code> tests value equality; <code>is</code> tests object identity.
<code>!=</code>	Inequality. Returns True if the operands are not equal.
<code>></code>	Greater Than. Returns True if the left operand is strictly greater than the right.
<code><</code>	Less Than. Returns True if the left operand is strictly less than the right.
<code>>=</code>	Greater Than or Equal To. Returns True if the left operand is greater than or equal to the right.
<code><=</code>	Less Than or Equal To. Returns True if the left operand is less than or equal to the right.

1.6.1 Logical Operators

Logical operators combine multiple Boolean expressions into a single, more complex condition.

Operator	Behaviour and Truth Table Summary
<code>and</code>	Returns True only if BOTH operands are True. Short-circuits: if the first operand is False, the second is not evaluated.
<code>or</code>	Returns True if AT LEAST ONE operand is True. Short-circuits: if the first operand is True, the second is not evaluated.
<code>not</code>	Unary operator. Reverses the Boolean value of its operand. <code>not True</code> → False; <code>not False</code> → True.

1.7 The Boolean Data Type

The `bool` type represents logical truth values. It has exactly two possible values: `True` and `False`. Booleans are the native result type of all comparison and logical operations. In Python, `bool` is a subclass of `int`: `True` behaves identically to `1` and `False` behaves identically to `0` in arithmetic contexts.

Key Concept: Truthy and Falsy Values

Any Python object can be evaluated in a Boolean context. Values that evaluate to **False** are called **falsy**: `0` , `0.0` , `""` (empty string), `None` , `[]` (empty list), `{}` (empty dict/set), `()` (empty tuple). All other values are **truthy**. This allows concise patterns like `if my_list:` to check whether a list is non-empty.

1.8 Lists

A **list** is Python's most versatile and commonly used built-in data structure. It is an **ordered**, **mutable**, and **heterogeneous** collection of elements, enclosed in square brackets and separated by commas. Ordered means elements maintain their insertion order. Mutable means elements can be added, removed, or modified after creation. Heterogeneous means a single list can contain elements of different data types.

1.8.1 List Indexing

Every element in a list is assigned an integer **index** representing its position. Python uses **zero-based indexing**: the first element has index `0` , the second has index `1` , and so on. Python also supports **negative indexing**: index `-1` refers to the last element, `-2` to the second-to-last, etc. Accessing an index outside the valid range raises an **`IndexError`** .

1.8.2 Key List Methods

Lists provide a rich set of built-in methods for modification and traversal:

Method / Operation	Description & Behaviour
<code>list.append(x)</code>	Adds element <code>x</code> to the END of the list. Modifies the list in place. $O(1)$ time complexity.
<code>list.insert(i, x)</code>	Inserts element <code>x</code> at position <code>i</code> . All subsequent elements shift right.
<code>list.remove(x)</code>	Removes the FIRST occurrence of element <code>x</code> by value. Raises <code>ValueError</code> if not found.
<code>del list[i]</code>	Removes the element at index <code>i</code> . Can also remove a slice: <code>del list[1:3]</code> .
<code>list.pop(i)</code>	Removes and RETURNS the element at index <code>i</code> (default: last element). Useful for stack operations.
<code>list.sort()</code>	Sorts the list in ascending order in place. Accepts <code>key</code> and <code>reverse</code> parameters.
<code>list.reverse()</code>	Reverses the list in place.

<code>len(list)</code>	Returns the number of elements in the list.
<code>list[a:b]</code>	Slicing: returns a new list from index a up to (not including) index b.

1.9 Loops

Loops are one of the most powerful constructs in programming — they allow a block of code to be executed **repeatedly**, either a fixed number of times or until a condition is met. Without loops, processing a dataset of thousands of records would require thousands of lines of identical code. Python provides two types of loops: the `for` loop and the `while` loop.

1.9.1 The for Loop

The `for` loop **iterates over an iterable** — any object that yields its elements one at a time, such as a list, string, tuple, dictionary, or range. On each iteration, the loop assigns the current element to a temporary variable (the **loop variable**) and executes the loop body. The loop terminates automatically when all elements have been processed.

1.9.2 The range() Function

The `range()` function generates an immutable sequence of integers, most commonly used to drive `for` loops a fixed number of times. It accepts up to three arguments: `range(start, stop, step)`. The `start` parameter defaults to 0, and `step` defaults to 1. The **stop value is exclusive** — it is not included in the generated sequence. `range(5)` generates 0, 1, 2, 3, 4.

1.9.3 The while Loop

The `while` loop continues executing its body as long as a given condition remains `True`. It is used when the number of iterations is not known in advance — for example, reading user input until a valid value is entered. A **crucial discipline** with `while` loops is ensuring the condition will eventually become `False` — failing to do so creates an infinite loop that freezes the program.

1.10 Dictionaries

A Python **dictionary** (`dict`) is an unordered (insertion-ordered from Python 3.7+) collection of **key-value pairs**. Each key maps uniquely to a value — dictionaries are essentially hash maps, providing average $O(1)$ lookup, insertion, and deletion performance. Keys must be **immutable** (strings, integers, or tuples), and must be unique within a dictionary. Values can be any Python object, including nested dictionaries or lists.

1.10.1 Key Dictionary Methods

Method	Description
<code>dict[key]</code>	Accesses the value for key. Raises <code>KeyError</code> if key does not exist.
<code>dict.get(key, default)</code>	Safely retrieves value for key. Returns default (<code>None</code> if not specified) instead of raising <code>KeyError</code> .
<code>dict.keys()</code>	Returns a view object of all keys in the dictionary.
<code>dict.values()</code>	Returns a view object of all values in the dictionary.
<code>dict.items()</code>	Returns a view object of all (key, value) pairs — essential for iterating over a dict.
<code>del dict[key]</code>	Removes the key-value pair for the specified key. Raises <code>KeyError</code> if key not found.
<code>dict.update(other)</code>	Merges another dictionary or iterable of key-value pairs into the dict, overwriting existing keys.
<code>key in dict</code>	Membership test. Returns <code>True</code> if key exists in the dictionary. $O(1)$ performance.

1.11 Sets

A **set** is an **unordered** collection of **unique** elements. Sets are defined using curly braces or the `set()` constructor. Because sets are implemented as hash tables, they provide $O(1)$ average-case membership testing — far faster than searching a list. Crucially, **duplicate elements are automatically discarded** upon insertion, making sets the most efficient tool for deduplication tasks. Sets are **mutable**, but their elements must be immutable (no lists or dicts as elements).

1.11.1 Set Operations

Sets support the full complement of classical mathematical set operations:

Operation	Method / Operator & Description
Union	<code>A B</code> or <code>A.union(B)</code> — All elements from both sets, with duplicates removed.
Intersection	<code>A & B</code> or <code>A.intersection(B)</code> — Only elements present in BOTH sets.
Difference	<code>A - B</code> or <code>A.difference(B)</code> — Elements in A that are NOT in B.
Symmetric Difference	<code>A ^ B</code> or <code>A.symmetric_difference(B)</code> — Elements in either A or B, but NOT in both.

Add element	<code>A.add(x)</code> — Inserts element <code>x</code> into the set. No-op if <code>x</code> already present.
Remove element	<code>A.remove(x)</code> — Removes <code>x</code> ; raises <code>KeyError</code> if not found. Use <code>A.discard(x)</code> for a safe version.
Membership test	<code>x in A</code> — Returns <code>True</code> if <code>x</code> is an element of set <code>A</code> . $O(1)$ complexity.

Data Cleaning Use Case

In Data Science, sets are invaluable for **removing duplicates** from datasets. For example, if `ids` is a list with repeated values, `unique_ids = list(set(ids))` produces a deduplicated list in a single expression. Sets are also essential for computing the **vocabulary** of a text corpus — the unique set of all words across all documents.

1.12 Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around **objects** — entities that combine **data (attributes)** and **behaviour (methods)** into a single unit. OOP promotes modularity, reusability, and the modelling of real-world entities in code. In Python, **everything is an object** — integers, strings, lists, and functions are all objects with attributes and methods.

1.12.1 Classes and Objects

A **class** is a blueprint or template that defines the structure and behaviour of objects. An **object** (also called an **instance**) is a concrete realization of a class, created by calling the class name as a constructor. Multiple objects can be created from the same class, each maintaining its own independent state.

1.12.2 The `__init__` Constructor

The `__init__()` method is a special **dunder (double-underscore)** method that Python calls automatically when a new object is instantiated. It is used to initialise the object's attributes with initial values. The first parameter of every instance method in Python is `self`, which is a reference to the instance being operated on. When calling `Person("Alice", 30)`, Python internally calls `Person.__init__(new_instance, "Alice", 30)`.

1.12.3 Inheritance

Inheritance is a mechanism by which a **child class** (subclass) acquires the attributes and methods of a **parent class** (superclass). This promotes code reuse and models 'is-a' relationships — a `Dog` is an `Animal`. The child class is defined by passing the parent class name in

parentheses: `class Dog(Animal):` . The child class can extend the parent with new attributes/methods and can override parent methods to provide specialized behaviour.

1.12.4 Method Overriding

Method overriding allows a child class to **redefine** a method that it has inherited from its parent class. The overriding method must have the **same name** as the parent's method. When the method is called on a child class object, Python executes the child's version. This is a key mechanism enabling **polymorphism** — the ability for different types of objects to respond differently to the same method call.

1.13 Exception Handling

An **exception** is an error that occurs during program **execution (runtime)** — after the code has passed syntactic validation. Exceptions interrupt the normal flow of execution. If an exception is not **handled**, it propagates up the call stack until it either reaches a handler or crashes the program with a traceback. Python's exception handling mechanism allows programs to anticipate, catch, and respond to errors gracefully, enabling the program to continue running or shut down cleanly.

1.13.1 Common Built-in Exceptions

Exception	Trigger Condition
<code>ValueError</code>	Raised when a function receives an argument of the correct type but an inappropriate value (e.g., <code>int("hello")</code>).
<code>TypeError</code>	Raised when an operation is applied to an object of an inappropriate type (e.g., <code>"hello" + 5</code>).
<code>KeyError</code>	Raised when a dictionary key is not found (e.g., <code>my_dict["missing_key"]</code>).
<code>IndexError</code>	Raised when a sequence index is out of range (e.g., <code>my_list[100]</code> on a 3-element list).
<code>ZeroDivisionError</code>	Raised when division or modulo by zero is attempted (e.g., <code>10 / 0</code>).
<code>FileNotFoundError</code>	Raised when an attempt to open a file fails because the specified path does not exist.
<code>AttributeError</code>	Raised when an attribute reference or assignment fails (object has no such attribute).
<code>NameError</code>	Raised when a local or global name is not found (variable used before assignment).

1.13.2 try-except-finally

The **try - except - finally** construct is Python's structured exception handling mechanism. Code that might raise an exception is placed inside the **try** block. If an exception occurs, execution jumps to the matching **except** block. The **finally** block, if present, **always executes** — whether or not an exception occurred — making it ideal for cleanup operations like closing file handles or releasing database connections.

1.13.3 Raising Exceptions with raise

The **raise** statement allows you to **manually trigger an exception**. This is used to signal that a function's preconditions have not been met or that an invalid state has been encountered. You can raise any built-in exception with a custom message, or define and raise custom exception classes (subclasses of **Exception**) for domain-specific error handling.

SECTION 2 — PRACTICAL & CODING REFERENCE

This section walks through every concept from Section 1 with complete, annotated code cells. Each cell includes the code, a conceptual explanation, the rationale for the approach, and a detailed output walkthrough.

Cell 1 — Exploring Python Data Types

# Declaring variables of each core data type		
age	= 25	# int: whole number
temperature	= 36.6	# float: decimal number
name	= "Alice"	# str: text value
is_active	= True	# bool: logical value
# Inspecting data types at runtime		
print(type(age))		# <class 'int'>
print(type(temperature))		# <class 'float'>
print(type(name))		# <class 'str'>
print(type(is_active))		# <class 'bool'>
# Checking bool is a subclass of int		
print(isinstance(True, int))		# True
print(True + True)		# 2 (True == 1)

Concept

The built-in `type()` function returns the class of any object. In Python, every value — including literals like `25` or `True` — is an instance of a class. The `isinstance()` function checks whether an object is an instance of a given class or its subclasses, confirming here that `bool` inherits from `int`.

Why This Approach?

Using `type()` for runtime introspection is a foundational debugging skill. In Data Science notebooks, it is common to lose track of whether a value is an `int` or `float` after multiple transformations. Checking types prevents silent errors — for instance, passing a `float` where a function expects an `int` can produce unexpected results.

Expected Output & Explanation

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
True (bool is a subclass of int)
2 (True + True = 1 + 1 = 2, demonstrating bool's integer nature)
```

Cell 2 — Declaring and Using Variables

```
# Assign and use variables
product_name = "Laptop"
price        = 75000
discount_pct = 10.5
in_stock     = True

# Variables make calculations readable
discount_amount = price * (discount_pct / 100)
final_price     = price - discount_amount

print(f"Product : {product_name}")
print(f"Original Price : Rs.{price}")
print(f"Discount : {discount_pct}% = Rs.{discount_amount}")
print(f"Final Price : Rs.{final_price}")
```

```
# Variables are mutable – values can change  
price = 70000    # price updated  
print(f"Updated Price : Rs.{price}")
```

Concept

Variables serve as named memory locations. The **f-string** syntax (`f"...{variable}..."`) introduced in Python 3.6 is the most readable and modern way to embed variable values directly inside strings. The expression inside `{ }` is evaluated at runtime and converted to a string.

Why This Approach?

Descriptive variable names (`product_name` , `discount_pct`) make code self-documenting. In a Data Science context, you might have `feature_mean` , `train_accuracy` , `learning_rate` — names that communicate purpose. Re-assigning `price = 70000` demonstrates mutability: unlike constants in other languages, Python variables can always be reassigned to a new value.

Expected Output & Explanation

```
Product : Laptop  
Original Price : Rs.75000  
Discount : 10.5% = Rs.7875.0  
Final Price : Rs.67125.0  
Updated Price : Rs.70000
```

Notice that `discount_amount` and `final_price` become floats because `discount_pct` is a float — Python automatically promotes the result of mixed int/float arithmetic.

Cell 3 — Interactive Input and Output

```
# Basic input – always returns a string  
name = input("Enter your name: ")  
print("Hello,", name)  
  
# print() with sep and end parameters  
print("Python", "Data", "Science", sep=" | ")  
print("Loading", end="...")  
print("Done")
```

```
# Formatted output with f-strings
city = "Mumbai"
temp = 34.5
print(f"Current temperature in {city}: {temp:.1f}°C")
```

Concept

The `input()` function displays the prompt string and suspends execution until the user presses Enter. The **entire typed value is captured as a string**. The `print()` function's `sep` parameter controls the separator between multiple arguments (default: space). The `end` parameter controls what appears after the last argument (default: newline). In the f-string, `{temp:.1f}` formats the float to 1 decimal place.

Why This Approach?

Understanding that `input()` always returns a string is critical. Beginners often assume numeric input is automatically an integer — this causes `TypeError` when attempting arithmetic. The `sep` and `end` parameters are widely used in CLI tools and logging utilities to control output formatting precisely.

Expected Output & Explanation

Enter your name: Alice (user types this)
Hello, Alice
Python | Data | Science (`sep='|'` replaces default space)
Loading...Done (`end='...'` suppresses newline, so Done prints on same line)
Current temperature in Mumbai: 34.5°C (`:.1f` formats to 1 decimal)

Cell 4 — Type Conversion in Practice

```
# User input is always a string – convert before arithmetic
age_str = input("Enter your age: ")    # e.g., user types '28'
age_int = int(age_str)
birth_year = 2025 - age_int
print(f"Approximate birth year: {birth_year}")

# float conversion
weight_str = "72.5"
weight_float = float(weight_str)
```

```
bmi_input_height = 1.75
bmi = weight_float / (bmi_input_height ** 2)
print(f"BMI: {bmi:.2f}")

# str conversion for concatenation
score = 95
message = "Your score is: " + str(score) + " / 100"
print(message)

# Safe conversion with try-except
raw = "hello"
try:
    num = int(raw)
except ValueError:
    print(f"Cannot convert '{raw}' to int")
```

Concept

Type conversion is a bridge between the string world of `input()` and the numeric world of computation. `int()` parses a string as a whole number. `float()` parses a string as a decimal. `str()` converts any value to its string representation. In the BMI example, `**` is Python's exponentiation operator (`1.75 ** 2 = 3.0625`). The `try-except` block around `int("hello")` prevents a `ValueError` from crashing the program.

Why This Approach?

Never trust raw user input. Always convert and validate. In production Data Science pipelines, raw data from CSV files or APIs often arrives as strings. The pattern `int()` / `float()` inside a `try-except` is the safe, professional way to handle uncertain data. The `{bmi:.2f}` format specifier rounds to 2 decimal places in the output, keeping display clean.

Expected Output & Explanation

Enter your age: 28 (user types 28)
Approximate birth year: 1997
BMI: 23.67
Your score is: 95 / 100
Cannot convert 'hello' to int (safe conversion handling in action)

Cell 5 — Conditional Logic — if / elif / else

```
# Simple if statement
temperature = 38.5
if temperature > 37.5:
    print("Fever detected. Please rest.")

# if-elif-else chain - grade classification
marks = 72

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 40:
    grade = "D"
else:
    grade = "F"

print(f"Marks: {marks} → Grade: {grade}")

# Nested conditional
age = 20
has_id = True
if age >= 18:
    if has_id:
        print("Entry permitted.")
    else:
        print("Valid ID required.")
else:
    print("Entry denied: underage.")
```

Concept

Python evaluates `if - elif - else` chains **sequentially from top to bottom**. As soon as one condition evaluates to `True`, its block executes and the remaining conditions are skipped. Indentation with exactly 4 spaces (or consistent tab width) is mandatory — Python uses whitespace as a syntactic delimiter, not curly braces. Nesting an `if` inside another `if` allows compound decision logic.

Why This Approach?

This pattern — an `if-elif-else` chain with ordered ranges — is extremely common in data classification tasks (binning numerical values into categories). The grade classifier mirrors the logic used in ML label encoding, where continuous values are mapped to discrete classes. Nesting conditions models real-world access control and business rules. Note that `marks = 72` matches `elif marks >= 60` and outputs Grade C, while all higher conditions are skipped.

Expected Output & Explanation

Fever detected. Please rest. (`38.5 > 37.5` is True)
Marks: 72 → Grade: C (`72 >= 60` is the first True condition)
Entry permitted. (`age=20 >= 18 AND has_id=True`)

Cell 6 — Comparison and Logical Operators

```
# Comparison operators
x, y = 15, 20
print(x == y)    # False
print(x != y)    # True
print(x < y)     # True
print(x >= 15)   # True

# Logical operators
age = 25
income = 50000

# and — both must be True
eligible_loan = (age >= 21) and (income >= 30000)
print(f"Loan eligible: {eligible_loan}")    # True

# or — at least one must be True
```

```
has_degree = False
has_experience = True
qualified = has_degree or has_experience
print(f"Qualified: {qualified}")          # True

# not — reverses Boolean
is_weekend = False
print(f"Is weekday: {not is_weekend}")    # True

# Chained comparisons (Pythonic)
score = 72
print(60 <= score < 80)                  # True — between 60 and 80
```

Concept

Comparison operators evaluate to **True** or **False** and form the atomic building blocks of conditions. Logical operators **and**, **or**, **not** combine these atomic conditions into complex expressions. A unique feature of Python is **chained comparisons**: `60 <= score < 80` is valid Python and is equivalent to `(score >= 60) and (score < 80)` — more concise and mathematically natural.

Why This Approach?

The loan eligibility check is a direct representation of real business logic: both conditions must hold (**and**). The qualification check allows either condition (**or**), matching a flexible hiring criterion. Chained comparisons are widely used in data filtering — checking if a value falls within a range — and are far more readable than their explicit **and**-connected equivalents. Short-circuit evaluation (**and** stops at first **False**, **or** stops at first **True**) also improves performance.

Expected Output & Explanation

```
False, True, True, True (comparison results)
Loan eligible: True (25 >= 21 AND 50000 >= 30000)
Qualified: True (has_experience is True)
Is weekday: True (not False = True)
True (60 <= 72 < 80, chained comparison)
```

Cell 7 — Creating and Manipulating Lists

```
# Creating a list
fruits = ["apple", "banana", "cherry", "date"]

# Indexing
print(fruits[0])    # apple    (first element)
print(fruits[-1])   # date     (last element)

# Slicing: fruits[start:stop] – stop is exclusive
print(fruits[1:3])  # ['banana', 'cherry']

# Modifying a list
fruits.append("elderberry")
print(fruits)       # adds to end

fruits.insert(1, "avocado")
print(fruits)       # inserts at index 1

fruits.remove("banana")
print(fruits)       # removes by value

del fruits[0]
print(fruits)       # removes by index

# List of mixed types – valid in Python
mixed = [42, 3.14, "hello", True, [1, 2]]
print(len(mixed))   # 5

# Iterating over a list
scores = [88, 92, 75, 95, 60]
total = 0
for score in scores:
    total += score
print(f"Average: {total / len(scores):.1f}")
```

Concept

Lists are zero-indexed and support both positive (forward) and negative (reverse) indexing. **Slicing** with `[start:stop]` returns a new list — the `stop` index is **not included**. `append()` is $O(1)$ and preferred for adding to the end. `insert()` is $O(n)$ as it shifts elements. `remove()` deletes the first matching value. `del` removes by index. The loop accumulates a running total — the `+=` shorthand updates `total` on each iteration.

Why This Approach?

Lists are the workhorse of Python data science. Storing dataset rows, model predictions, feature values — all naturally fit into lists. The average calculation pattern (iterate, accumulate, divide by length) is fundamental and mirrors what libraries like NumPy do under the hood. Understanding slice notation is essential since it directly translates to array slicing in NumPy and DataFrame row selection in pandas.

Expected Output & Explanation

```
apple (index 0)
date (index -1, last element)
['banana', 'cherry'] (slice [1:3])
['apple', 'banana', 'cherry', 'date', 'elderberry'] (after append)
['apple', 'avocado', 'banana', 'cherry', 'date', 'elderberry'] (after insert)
['apple', 'avocado', 'cherry', 'date', 'elderberry'] (banana removed)
['avocado', 'cherry', 'date', 'elderberry'] (index 0 deleted)
5 (len of mixed list)
Average: 82.0 (sum=410, divided by 5)
```

Cell 8 — for Loops and the range() Function

```
# Iterating over a list
cities = ["Delhi", "Mumbai", "Bangalore", "Chennai"]
for city in cities:
    print(f" Processing: {city}")

# range(stop): 0 to stop-1
for i in range(5):
    print(i, end=" ")    # 0 1 2 3 4
print()    # newline

# range(start, stop): custom start
```

```
for i in range(1, 6):
    print(i, end=" ")    # 1 2 3 4 5
print()

# range(start, stop, step): custom step
for i in range(0, 20, 5):
    print(i, end=" ")    # 0 5 10 15
print()

# Enumerate: get index AND value simultaneously
languages = ["Python", "R", "SQL", "Scala"]
for index, lang in enumerate(languages, start=1):
    print(f" {index}. {lang}")

# While loop – unknown iteration count
attempts = 0
max_attempts = 3
while attempts < max_attempts:
    print(f" Attempt {attempts + 1}")
    attempts += 1
```

Concept

The **for** loop binds each element of the iterable to the loop variable in turn. **range(stop)** generates integers from 0 up to (not including) **stop**. **range(start, stop, step)** gives full control. **enumerate()** wraps an iterable and yields **(index, value)** pairs — the **start** parameter sets the initial index value (default 0). The **while** loop runs until **attempts < max_attempts** is **False**, which occurs when **attempts** reaches 3.

Why This Approach?

In data processing, **for city in cities** is more Pythonic and readable than **for i in range(len(cities)):**. However, when you need both the element and its index, **enumerate()** is the correct choice over manual index tracking. The **while** loop is ideal for retry logic (network requests, user input validation) where you don't know upfront how many attempts will be needed.

Expected Output & Explanation

Processing: Delhi / Mumbai / Bangalore / Chennai (one per line)
0 1 2 3 4 (range(5))
1 2 3 4 5 (range(1, 6))
0 5 10 15 (range(0, 20, 5) — step of 5)
1. Python / 2. R / 3. SQL / 4. Scala (enumerate with start=1)
Attempt 1 / Attempt 2 / Attempt 3 (while loop, 3 iterations)

Cell 9 — Dictionaries — Key-Value Data Storage

```
# Creating a dictionary
student = {
    "name"      : "Arjun",
    "age"       : 22,
    "gpa"       : 8.7,
    "subjects": ["ML", "Statistics", "Python"]
}

# Accessing values
print(student["name"])           # Arjun
print(student.get("email", "Not set")) # Not set (safe access)

# Adding and updating entries
student["email"] = "arjun@university.in"
student["gpa"]   = 9.1           # update existing
print(student["gpa"])           # 9.1

# Deleting an entry
del student["age"]

# Iterating over a dictionary
print("\n--- Student Profile ---")
for key, value in student.items():
    print(f"  {key:12s}: {value}")

# Dictionary of lists — common in data loading
dataset = {
```

```

    "feature1": [1.2, 3.4, 5.6],
    "feature2": [7.8, 9.0, 2.3],
    "label"    : [0, 1, 1]
}
print(f"Rows: {len(dataset['label'])}")    # 3

```

Concept

Dictionaries are accessed via keys (O(1) hash lookup). `student["email"]` raises **KeyError** if the key doesn't exist; `.get()` returns a default value instead, making it safe for uncertain keys. Assignment adds or updates a key. `.items()` yields **(key, value)** tuples, enabling clean iteration. The **{key:12s}** format specifier pads the key string to 12 characters for aligned output. The **dictionary of lists** pattern at the bottom mirrors the structure of a pandas DataFrame.

Why This Approach?

In Data Science, dictionaries are ubiquitous: model hyperparameters, configuration settings, column mappings, frequency counts. The `.get()` pattern is a professional defensive coding habit — always use it when accessing dictionary keys that may not exist. The **dictionary of lists** is the internal structure of a pandas DataFrame, so understanding how to build and iterate over it is direct preparation for pandas.

Expected Output & Explanation

```

Arjun (direct key access)
Not set (get() with default — 'email' key did not exist yet)
9.1 (GPA updated)
--- Student Profile ---
name      : Arjun
gpa       : 9.1
subjects  : ['ML', 'Statistics', 'Python']
email     : arjun@university.in (age was deleted)
Rows: 3 (3 labels in dataset)

```

Cell 10 — Sets — Unique Collections and Set Operations

```

# Creating a set - duplicates are silently removed
raw_ids = [101, 102, 101, 103, 102, 104]
unique_ids = set(raw_ids)
print(unique_ids)    # {101, 102, 103, 104}

```

```
print(len(unique_ids))    # 4

# Set operations – data cleaning example
enrolled = {"Alice", "Bob", "Charlie", "Diana"}
submitted = {"Bob", "Diana", "Eve"}

# Intersection: who submitted AND was enrolled?
valid = enrolled & submitted
print(f"Valid submissions : {valid}")

# Difference: enrolled but did NOT submit
missing = enrolled - submitted
print(f"Did not submit    : {missing}")

# Union: all names seen
all_names = enrolled | submitted
print(f"All names        : {all_names}")

# Add and remove
enrolled.add("Frank")
enrolled.discard("Alice") # safe remove
print(f"Updated enrolled : {enrolled}")

# Membership testing – O(1) vs O(n) for lists
print("Bob" in enrolled)  # True
```

Concept

When `set(raw_ids)` is called on the list `[101, 102, 101, 103, 102, 104]`, the duplicates (101 and 102) are automatically discarded. The set operators `&` (intersection), `-` (difference), and `|` (union) provide concise mathematical set logic. `discard()` is the safe counterpart to `remove()` — it silently does nothing if the element is not present, avoiding a `KeyError`.

Why This Approach?

This cell demonstrates a common data quality workflow: remove duplicate IDs, find who submitted, identify missing submissions. These are daily operations in data pipelines. The

membership test `"Bob" in enrolled` is $O(1)$ for a set vs $O(n)$ for a list — for large datasets (millions of records), this performance difference is significant. Sets are the correct data structure when uniqueness matters and order does not.

Expected Output & Explanation

`{101, 102, 103, 104}` (duplicates removed from `raw_ids`)

4 (4 unique IDs)

Valid submissions : `{'Bob', 'Diana'}` (in both sets)

Did not submit : `{'Alice', 'Charlie'}` (in enrolled, not in submitted)

All names : `{'Alice', 'Bob', 'Charlie', 'Diana', 'Eve'}` (union)

Updated enrolled : `{'Bob', 'Charlie', 'Diana', 'Frank'}` (Alice removed, Frank added)

True (Bob is in enrolled)

Cell 11 — Object-Oriented Programming — Classes, Inheritance, and Overriding

```
# Base class (Parent)
class Animal:
    def __init__(self, name, species):
        self.name = name # attribute: state
        self.species = species

    def speak(self): # method: behaviour
        return f"{self.name} makes a sound."

    def describe(self):
        return f"{self.name} is a {self.species}."

# Child class — inherits from Animal
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "Canis familiaris") # call parent init
        self.breed = breed # additional attribute

    def speak(self): # method overriding
```

```
        return f"{self.name} says: Woof!"

    def fetch(self):                # new method unique to Dog
        return f"{self.name} fetches the ball!"

# Another child class
class Cat(Animal):
    def speak(self):
        return f"{self.name} says: Meow!"

# Instantiating objects
generic = Animal("Generic", "Unknown")
dog      = Dog("Bruno", "Labrador")
cat      = Cat("Whiskers", "Felis catus")

# Accessing attributes and calling methods
print(generic.speak())
print(dog.speak())                # overridden method
print(cat.speak())                # overridden method
print(dog.describe())            # inherited from Animal
print(dog.fetch())                # Dog-specific method

# Polymorphism: same interface, different behaviour
animals = [generic, dog, cat]
for animal in animals:
    print(animal.speak())          # calls the right version for each type
```

Concept

The **Animal** class defines two attributes (**name** , **species**) in **__init__()** and two methods (**speak** , **describe**). **Dog** inherits from **Animal** using **class Dog(Animal)** . The **super().__init__()** call invokes the parent class's constructor, initialising inherited attributes. **Dog** overrides **speak()** with its own version. The polymorphism loop at the bottom calls **speak()** on objects of three different types — Python automatically dispatches to the correct version based on each object's actual type.

Why This Approach?

OOP is foundational to Data Science frameworks. Scikit-learn models are classes: you `fit()` them, `predict()` with them. PyTorch neural networks are classes inheriting from `nn.Module` with a `forward()` method. Understanding OOP lets you extend, customize, and build upon these frameworks. Polymorphism (the for loop at the bottom) is what allows `sklearn` pipelines to call `fit()` on any estimator — the interface is the same regardless of the underlying algorithm.

Expected Output & Explanation

Generic makes a sound.

Bruno says: Woof! (Dog's overridden speak)

Whiskers says: Meow! (Cat's overridden speak)

Bruno is a Canis familiaris. (inherited describe() from Animal)

Bruno fetches the ball! (Dog-specific method)

Generic makes a sound. (polymorphism loop — `Animal.speak`)

Bruno says: Woof! (polymorphism loop — `Dog.speak`)

Whiskers says: Meow! (polymorphism loop — `Cat.speak`)

Cell 12 — Exception Handling — try, except, finally, and raise

```
# --- Basic try-except ---
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        print(" Error: Cannot divide by zero!")
        return None

print(safe_divide(10, 2))    # 5.0
print(safe_divide(10, 0))    # Error handled

# --- Multiple except clauses ---
def parse_number(value):
    try:
```

```
        return int(value)
    except ValueError:
        print(f"  ValueError: '{value}' is not a valid integer.")
    except TypeError:
        print(f"  TypeError: Unsupported type {type(value).__name__}.")
    return None

parse_number("42")    # works - returns 42
parse_number("abc")   # ValueError
parse_number(None)    # TypeError

# --- try-except-finally: resource management ---
def read_config(filename):
    file = None
    try:
        file = open(filename, "r")
        data = file.read()
        print(f"  Read {len(data)} characters.")
    except FileNotFoundError:
        print(f"  File '{filename}' not found.")
    finally:
        if file:
            file.close()
            print("  File handle closed.")
        print("  Cleanup complete.")

read_config("config.txt")

# --- Raising exceptions manually ---
def set_age(age):
    if not isinstance(age, int):
        raise TypeError("Age must be an integer.")
    if age < 0 or age > 150:
```

```
        raise ValueError(f"Invalid age: {age}. Must be 0-150.")
    return age

try:
    print(set_age(25))      # valid
    print(set_age(-5))     # raises ValueError
except ValueError as e:
    print(f"  Caught: {e}")
```

Concept

The **try** block wraps potentially failing code. When an exception occurs, Python searches for a matching **except** clause (matched by exception class). **finally** always runs — even if **return** is called inside **try** — making it the correct place for cleanup code. The **as e** syntax binds the exception instance to a variable, allowing you to print its message. **raise** with a specific exception class and message signals caller code that a precondition was violated. **isinstance()** in **set_age()** checks the type before conversion, preventing downstream errors.

Why This Approach?

Robust exception handling is the difference between a prototype and a production system. In a data pipeline, a **FileNotFoundError** on one file should log the error and continue — not crash the entire run. The **finally** block is critical for database connections, file handles, and network sockets — resources that must be released regardless of success or failure. The **raise** pattern in **set_age()** is a standard technique for building APIs and model interfaces: validate inputs at the boundary and raise descriptive exceptions immediately, rather than letting invalid data silently corrupt downstream computations.

Expected Output & Explanation

```
5.0 (10 / 2)
Error: Cannot divide by zero! (ZeroDivisionError caught)
None (safe_divide returned None)
ValueError: 'abc' is not a valid integer. (caught ValueError)
TypeError: Unsupported type NoneType. (caught TypeError)
File 'config.txt' not found. (FileNotFoundError caught)
Cleanup complete. (finally runs regardless)
25 (set_age(25) valid)
Caught: Invalid age: -5. Must be 0-150. (ValueError raised and caught)
```

Quick Reference — Python Fundamentals

Concept	Key Syntax / Pattern
Core Data Types	<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> — check with <code>type()</code>
Variable Assignment	<code>variable_name = value</code> — use snake_case
Input / Output	<code>x = input("prompt")</code> / <code>print(f"Value: {x}")</code>
Type Conversion	<code>int()</code> , <code>float()</code> , <code>str()</code> , <code>bool()</code>
Conditional Logic	<code>if cond:</code> / <code>elif cond:</code> / <code>else:</code>
Comparison Ops	<code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>
Logical Ops	<code>and</code> , <code>or</code> , <code>not</code>
Lists	<code>[a, b, c]</code> — mutable, ordered, zero-indexed
List Methods	<code>append()</code> , <code>insert()</code> , <code>remove()</code> , <code>del</code> , <code>pop()</code>
for Loop	<code>for item in iterable:</code> / <code>for i in range(n):</code>
while Loop	<code>while condition:</code> — ensure termination
Dictionaries	<code>{key: value}</code> — <code>.get()</code> , <code>.items()</code> , <code>.keys()</code>
Sets	<code>{a, b, c}</code> or <code>set()</code> — unique, <code> </code> , <code>&</code> , <code>-</code>
Class Definition	<code>class Name:</code> + <code>__init__(self, ...)</code> + methods
Inheritance	<code>class Child(Parent):</code> + <code>super().__init__()</code>
Exception Handling	<code>try:</code> / <code>except ExcType as e:</code> / <code>finally:</code>
Raise Exception	<code>raise ValueError("message")</code>

Chapter Summary

This chapter established a thorough command of Python's foundational building blocks. Each concept forms a direct stepping stone towards professional Data Science and GenAI development:

1. **Data Types & Variables** — The atomic building blocks of every program. Understanding `int`, `float`, `str`, `bool`, and how Python's dynamic typing works is prerequisite knowledge for all subsequent chapters.
2. **Input / Output & Type Conversion** — Essential for building interactive programs and safely processing real-world data, which almost always arrives as strings.
3. **Conditional Statements & Operators** — The decision-making engine of programs. The same logic powers feature engineering rules, model threshold conditions, and business logic in data applications.

4. **Lists** — The foundation of Python's data model and a direct precursor to NumPy arrays and pandas Series.
5. **Loops** — Enable batch processing of datasets and are the conceptual basis for understanding vectorised operations in NumPy.
6. **Dictionaries & Sets** — Efficient data lookup and deduplication. Dictionaries are the structural basis of pandas DataFrames; sets power vocabulary extraction and data cleaning tasks.
7. **OOP: Classes, Inheritance, Overriding** — The architectural paradigm of Python's entire ML ecosystem (scikit-learn, PyTorch, TensorFlow). Deep understanding of OOP is mandatory for advanced framework usage.
8. **Exception Handling** — The difference between fragile prototypes and production-grade pipelines. Robust error handling makes systems reliable and debuggable.

What's Next?

In the next chapter, we will build on these foundations to explore Python's standard library, functional programming features (list comprehensions, lambda functions, map/filter), and advanced data structures including tuples, named tuples, and the collections module — all of which are widely used in professional Data Science codebases.